

졸업 프로젝트 9월 2차 격주보고서		
SNN을 활용한 고효율 추천 시스템 개발 및 최적화 연구		
2020058259 이승현 2020061676 이강혁		지도교수 김상욱 교수님
지난 2주간 진행사항	✓ ANN to SNN 변환을 위한 ANN 데이터셋 선행 작업 ✓ SNN 모델 인코딩/디코딩 알고리즘 수정 ✓ 시연 준비	△ ○ X
진행 내용	1. ANN to SNN 전처리 진행 중 이슈 해결 1) BN / Dropout / MaxPool 불일치 저희의 NCF 모델은 오직 Embedding -> Linear -> ReLU -> Linear 구조만 사용합니다. 따라서 BatchNorm, Dropout, MaxPool 같은 연산은 애초에 포함하지 않아 SNN 변환 시 불일치 문제 자체가 발생하지 않습니다. 2) 시간 정보 활용 모호 원본 Taobao 데이터셋은 timestamp만 제공하지만, time_bin =3600 옵션을 기본값으로 넣어둬서 자동으로 시간 binning이 적용되게끔 하였습니다. 3) 임베딩 내적 음수 처리문제 입력 데이터(클릭/구매)는 0또는 1 값이므로 음수가 없고, 모델 설계도 ReLU만 사용하여 hidden activation도 음수가 없습니다. 문제는 임베딩 내적인데, 이건 아직 코드에서 별도로 음수 보정은 하지 않았습니니다. 즉, 음수 문제는 아직 남아있지만 calibration 기반 보정을 통해 해결하려고 합니다.	

2. Calibration 기반 보정 채택

1) calibration 보정

calibration 보정이란 ANN 학습 후, Calibration dataset을 모델에 통과시켜 레이어별 출력 분포를 측정하고, 이를 바탕으로 스케일링·threshold를 보정하는 방식입니다. 이는 모델 구조 변경이 없어 파라미터 수가 증가하지 않고, 학습된 ANN 성능 그대로 유지 가능하다는 장점이 있습니다. 하지만 음수 정보 자체를 spike로 직접 표현하지는 못하고, 근사적으로 보정하는 방식입니다.

2) Positive/Negative Channel 방식

feature를 양수와 음수로 분리하여 두 채널로 spike를 발생시키는 방법입니다. 음수 정보까지 SNN에서 직접적으로 표현 가능하고 변환 오차가 줄어드는 장점이 있습니다.

하지만 단점으로 뉴런/시냅스 수가 2배로 증가하여 추천 시스템의 유저·아이템 규모가 큰 경우 자원 소모가 감당 불가하다는 점이 있습니다. 특히, SpiNNaker 같은 뉴로모픽 칩에서는 자원 제약이 심각해질 수 있습니다.

3) calibration 보정 채택 근거

첫번째로 규모 문제로, Positive/Negative channel 방식은 뉴런 수가 2배로 증가하여 대규모 추천 시스템 데이터셋에서는 자원 부담이 큼니다.

두번째로 Calibration 방식은 모델 구조를 바꾸지 않고도 threshold shifting과 scaling만으로 음수 처리를 근사할 수 있어 구현이 간단합니다.

본 연구의 목표는 ANN to SNN 변환의 효율성 검증이므로, 정확도 손실을 허용 가능한 수준에서 제어하는 것이 중요합니다. Calibration 기반 보정은 이러한 요구와 잘 맞는다고 생각합니다.

3. 2-pass split 방식을 도입

초대형 로그 데이터를 다루기 위해 chunksize를 사용하여 메모리에 안전하게 처리했습니다. 이 과정에서 user/item 매핑과 행동 로그 수집을 메모리 폭주 없이 수행할 수 있었습니다.

다만, 단순히 chunk마다 train/test를 분리하면 전체 데이터 분포가 왜곡될 수 있으므로, 2-pass split 방식을 도입하였습니다.

1차 패스에서 전체 구매 로그의 인덱스를 수집한 뒤 전역적으로 80:20 비율로 분리하고, 2차 패스에서 다시 chunk 단위로 읽으면서 train/test에 할당하였습니다. 이를 통해 메모리 효율성과 통계적 일관성을 동시에 달성하여, ANN과 SNN의 성능 비교가 공정하게 이뤄질 수 있도록 하였습니다.

4. 인코딩/디코딩 알고리즘 수정

1) Two-Tower 구조 적용

NCF와 비슷하게, 사용자(User Tower)와 아이템(Item Tower)을 독립적인 임베딩으로 처리하고, 이를 각각 SNN의 population으로 생성합니다.

- ✓ **메모리 효율성:** User-Item 매트릭스를 그대로 입력으로 사용하면, 첫 번째 레이어의 가중치 매트릭스의 크기가 매우 커지는 문제가 있습니다. 이는 layer 깊지 않은 SNN에서 더욱 비효율적이라 할 수 있습니다. Two-Tower 구조를 사용하면 각 입력에 대한 임베딩 테이블을 분리하여 유지할 수 있고, 이는 전체 파라미터 수를 줄이고 메모리 사용량을 줄입니다.
- ✓ **병렬 처리:** SpiNNaker는 다수의 독립적인 코어로 구성되어 병렬 컴퓨팅이 가능합니다. User Tower와 Item Tower 각각은 서로 종속성이 없기 때문에 각 타워를 병렬로 동시에 실행할 수 있습니다.

2) Temporal Coding 적용

뉴런의 첫 스파이크가 발생하는 시간(Time-to-First-Spike, TTFS)으로 정보를 인코딩합니다. 값이 클수록 더 빨리 스파이크가 발생하도록 하여, 짧은 시간으로도 정보 전달이 가능합니다.

	✓ 에너지 효율성: SpiNNaker 보드의 전력 소비는 대부분 스파이크를 생성하고 전송하는 과정(시냅스 이벤트)에서 발생합니다. Rate Coding은 하나의 값을 표현하기 위해 다수의 스파이크를 필요로 하지만, Temporal Coding은 뉴런당 단 하나의 스파이크만 사용합니다. 이는 전체 네트워크의 이벤트 발생 횟수를 감소시키고, 이를 통해 에너지 소모를 최소화하면서도 정보의 손실 없이 전달할 수 있습니다.. ✓ 추론 속도 향상: Rate Coding은 정확한 발화율을 측정하기 위해 정해진 시뮬레이션 시간동안 스파이크를 누적해야 합니다. 반면에 Temporal Coding은 출력 뉴런에서 첫 번째 스파이크가 발생하는 즉시 결과가 나오므로, 스파이크의 누적을 기다릴 필요 없이 바로 연산을 완료하고 결과를 도출할 수 있습니다. 이는 실시간 추론에 유리합니다. 3) 메모리 제한을 해결하기 위한 mini-batch 사용 EBRAINS에서는 메모리를 최대 2GB 제공합니다. 이에 맞추기 위해, 전체 데이터를 한 번에 처리하는 대신 관리 가능한 크기의 mini-batch로 나누어 처리합니다. ✓ 메모리 제약: EBRAINS가 제공하는 메모리의 제약 때문이 아니더라도, SpiNNaker 칩의 메모리는 한정되어 있기 때문에, 한정된 메모리를 사용하는 방법이 필요합니다. mini-batch를 사용하여 순차적으로 데이터를 처리하면 각 batch마다 SpiNNaker가 메모리에 구성해야 하는 네트워크의 크기가 작아지므로, 메모리 제한에 대한 문제를 해결할 수 있습니다.
진행 중 이슈	다만, EBRAINS에서 제공하는 쿼터, 5000 core-hours를 전부 사용하여 추가적인 쿼터를 요청하였습니다. 이 때문에 아직 수정한 코드의 테스트는 진행하지 못하였습니다. 쿼터가 제공되는대로 테스트를 진행할 예정입니다.
다음 2주간 계획	1. 테스트 성능 판단 및 피드백 2. 시연 준비